
Slides as Code

Source Tracking, Conversational Editing, and Change History
for AI-Generated Presentations

The Problem Today



The Feedback Loop is Broken

Generate a deck, review it, spot issues — now what?



Option A: Start Over

Re-run the full pipeline from scratch, losing any tweaks



Option B: Manual Editing

Edit the PPTX directly, leaving the code-generation paradigm entirely



Slow & Error-Prone

Each fix cycle disconnects code from output and compounds mistakes



Maximum Friction

The most common workflow (generate, review, fix, repeat) has the most friction

What Slides as Code Enables



Edit in Place

Modify the generating script directly — not the outline, not the PPTX. The code is where the edits happen.



Full Traceability

Every cataloged deck links back to its source code, engine, theme, and originating outline.



Portable History

Change logs travel inside the PPTX as speaker notes metadata — not locked in a database.

01

SECTION

How It Works

Architecture Overview

- Source material flows through /create-outline to a markdown outline
- **create-deck:** generates a JS or Python script — the source of truth
- The script produces a PPTX, which is cataloged with full source lineage
- **edit-deck:** feeds changes back into the script, not the outline

The Script Is the Source of Truth



Most Precise Representation

Contains exact positions, colors, fonts, and layout logic for every slide



Not Disposable Anymore

Today scripts are treated as build artifacts — slides-as-code makes them durable



Mutable & Trackable

Scripts can be edited, version-controlled, and diffed like any source code



Outline Becomes Lineage

The outline is tracked as metadata but never mutated after generation

The Closed Loop



02

SECTION

Source Tracking

Tracing Decks Back to Code

- Every aippt-generated deck gets linked to its source script in the catalog
- Know which script, engine, theme, and outline produced any deck
- Look up source with a single CLI command
- Existing decks from arbitrary PPTX files are unaffected — source fields are nullable

What Gets Tracked

Field

- Script Path — relative path to JS or Python file
- Engine — pptxgenjs or python-pptx
- Theme — theme name (amd, default, etc.)

Purpose

- Outline Path — the originating markdown outline
- Generated At — timestamp of last generation
- All fields nullable — only aippt-generated decks populate them

Auto-Detection



Engine Detection

Scans the first 50 lines of the script for `require('pptxgenjs')` or from `pptx` import to determine the engine.



Theme Detection

Finds `themes/name.yaml` references in the script and maps them to known theme configurations.



Zero Configuration

No manual flags needed — the ingest pipeline figures it out automatically from the script content.



Manual Override

When auto-detection isn't sufficient, override with explicit `--source` and `--theme` flags on the CLI.

03

SECTION

Conversational Editing

Meet /edit-deck



Natural Language Editing

A new Claude Code skill for editing decks through conversation



Script or Deck Name

Works with script paths or deck names from the catalog



Single-Slide Edits

"make slide 3 a numbered list" — quick, targeted changes



Deck-Level Changes

"move compliance section before security" — structural reorganization



Batch Operations

"add speaker notes to every slide" — apply changes across all slides at once

Edit Workflow



What You Can Edit

Single-Slide Edits

- Change layout type (bullet, two-column, numbered)
- Rewrite content and bullets
- Add or edit speaker notes
- Add or swap images
- Adjust styling (colors, fonts, spacing)

Deck-Level Edits

- Add, remove, or duplicate slides
- Reorder slides or entire sections
- Change theme or global styling
- Add section dividers
- Batch operations across all slides

Safety and Error Handling



Backup Before Editing

First edit creates a .bak file preserving the original state — only one backup, always the earliest pre-edit version.



Diff Preview

See exactly what changed in the script before writing. No surprises, full transparency.



File Lock Detection

Warns if the PPTX is open in PowerPoint on Windows before attempting to regenerate.



Error Recovery

If the edited script fails to run, offer to restore from backup or attempt an auto-fix.



Git-Friendly

Edited scripts are plain text that produces clean diffs for version control workflows.

04

SECTION

Change History

Metadata in Speaker Notes

Speaker notes content appears here as normal text.

```
--aippt-meta---
```

```
source: outline → pptxgen.js
```

```
created: 2026-03-11
```

```
layout: two_column
```

```
theme: default
```

```
history:
```

- 2026-03-11: Created from outline (bullet layout)
- 2026-03-11: Changed to two_column layout [/edit-deck]
- 2026-03-12: Rewrote bullets for executive audience [/edit-deck]

```
--end-aippt-meta---
```

History That Travels With the File



Embedded in PPTX

Change history lives in speaker notes, not locked in an external database



Portable by Default

Share the file via email, Teams, Slack — the history comes along automatically



LLM Context

The AI reads edit history before making changes to understand what's been tried



Self-Cleaning

Capped at 10 entries — oldest trimmed on write to keep notes manageable

05

SECTION

The Big Picture

User Journey



Competitive Landscape

Developer Slide Tools

- Slidev, Marp, reveal.js — own markdown-as-source
- Excellent developer experience
- No PPTX round-trip capability
- Cannot produce corporate-ready slide decks

AI Slide Tools

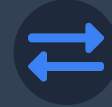
- Gamma, Beautiful.ai, PPTAgent — generate polished slides
- Impressive visual output from AI
- No human-readable source code
- No version control or diff-friendly format

What No Other Tool Does



Human-Readable Source

JS/Python scripts anyone can read, modify, and understand — not opaque binary formats.



PPTX Round-Trip

Generates real PowerPoint files, not HTML or images — opens in PowerPoint, Keynote, Google Slides.



Conversational Editing

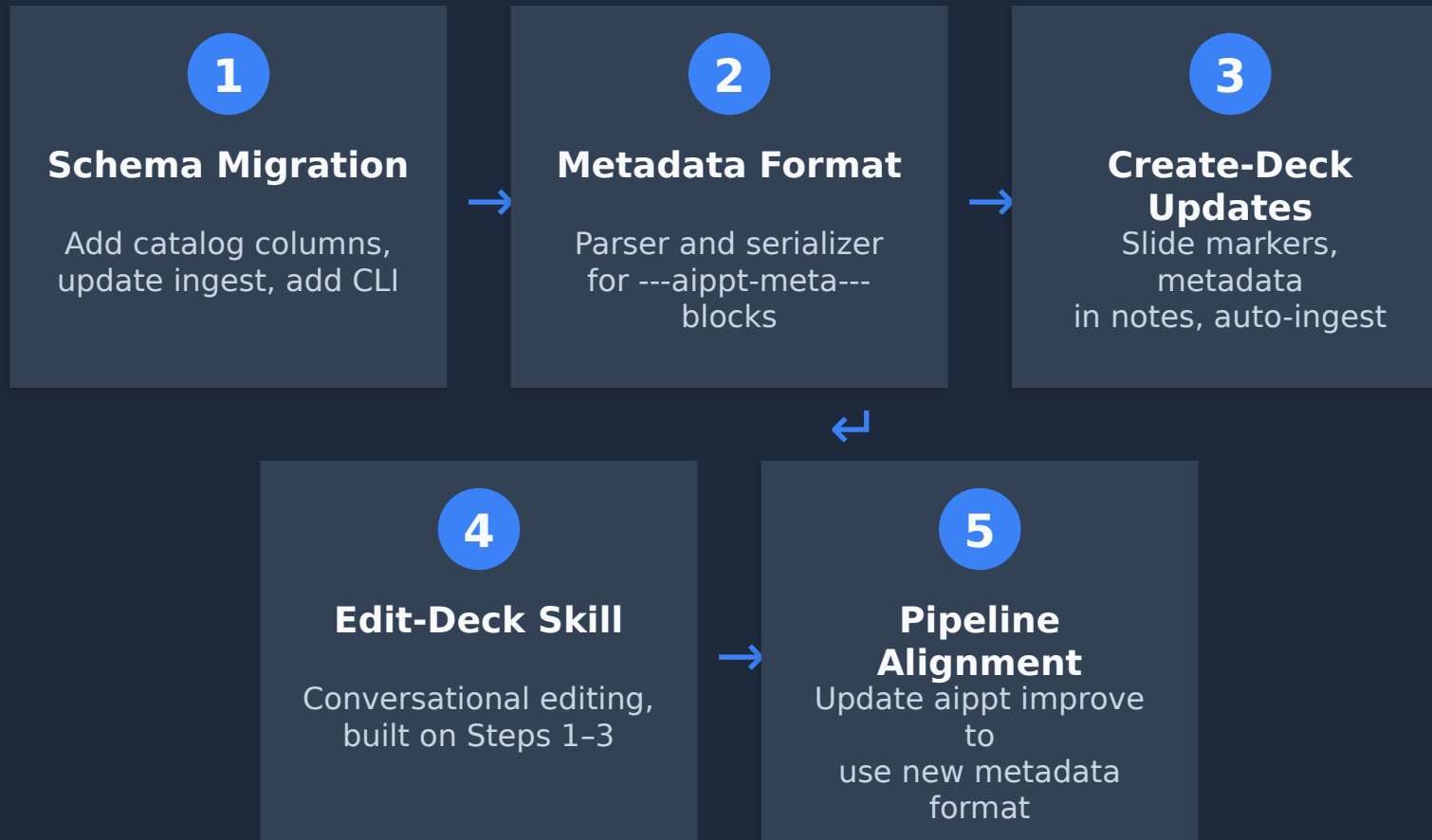
Fix slides through natural language against the source code — no manual PPTX surgery.



Diff & Version Tracking

Scripts are plain text, git-ready from day one. Every change produces a clean, reviewable diff.

Implementation Roadmap



06

SECTION

Closing

What's Next



Testing Strategy

Unit tests for metadata parsing, integration tests for source tracking, E2E tests for the full create-edit-review loop.



Future Extensions

Code-based analysis, reference enrichment, web UI for editing — extending the slides-as-code paradigm.



Research Validation

CMU's AutoPresent confirms: code-as-intermediate-representation produces higher quality AI slides than direct generation.

Slides as Code

- Generate decks from outlines with `/create-deck`
 - Review visually with `/deck-review`
- Fix issues conversationally with `/edit-deck`
- Every deck traceable back to its source code